

Tabular RL & Discrete MDPs

Session 1 - Fundamentals

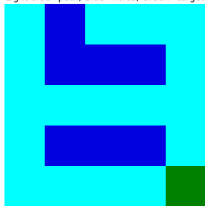
RL bootcamp 2026 · Salzburg 16-18 September



The mathematics behind the implementation

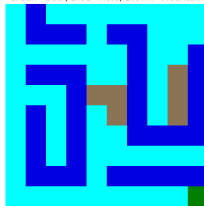
- the discrete MDP $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$
- state S : the state at which the agent moves in the environment
- actions A : right left, down, up
- transition probability P : conditional probability of moving to a next state given that the agent is in current state taking an action, computed through the step function `observation`
- rewards R : depends on the environment

Light blue=path, Blue=holes, Green=target



(a) Simple maze

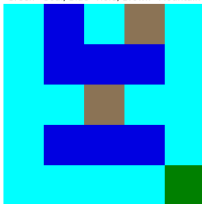
Green=Goal, Blue=Hole, Brown=Mountain



(b) Larger complex maze

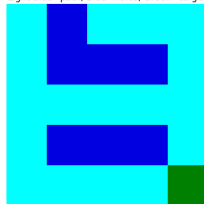
The mathematics behind the implementation

Green=Goal, Blue=Hole, Brown=Mountain



(a) Complex maze

Light blue=path, Blue=holes, Green=target



(b) Simple maze

- Q-learning (off-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [\underbrace{r + \gamma \max_{a_{\text{next}}} Q(s_{\text{next}}, a_{\text{next}})}_{\text{Bellman optimality equation}} - Q(s, a)]$$

- SARSA (on-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [\underbrace{r + \gamma Q(s_{\text{next}}, a_{\text{next}})}_{\text{Bellman expectation equation}} - Q(s, a)]$$

What to find in the code repository?

```
|— Environments      # Different environments
|   |— Simplest maze environment.ipynb
|   |— Simple maze environment.ipynb
|   |— Complex maze environment.ipynb
|   |— Larger complex maze environment.ipynb
|— Solutions        # Different Tabular RL methods used to solve the the optimal path to follow
|   |— Simplest maze solutions.ipynb
|   |— Simple maze solutions.ipynb
|   |— Complex maze solutions.ipynb
|   |— Larger complex maze solutions.ipynb
```

- 4 environments and 4 shortest paths solutions
- Jupyter notebooks
- ✈ More complex self-made environments will be presented in the remaining parts of the tutorial.

How is the solutions code organised?

The solutions are implemented in each code cells:

- run the baseline of the environment
- pre-run the functions to display the agent steps path and the animation in the environment
- training hyperparameter and epsilon-greedy policy
- Q-learning and SARSA implementation
 - Optimal path with Q-learning and SARSA with arrows shown in each environments
 - Q-tables (Q-learning and SARSA)
 - Environment with Q-values
 - Path animation of Q-learning and SARSA in each maze environments

Brief description on the environments construction

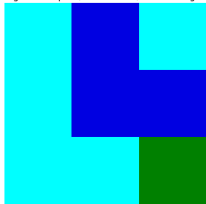
The code is based on the Gymnasium Documentation “*make your own custom environment*”

- `__init__`
- `reset`: initializes the environment size, number of steps, map, observation space and actions
- `step`: returns `observation`, `reward`, `terminated`, `truncated`, `info`
- additional functions used in the `step` implementation to define the `observation`, and `info` return values
- `render`: return a canvas with the grid of the maze using the `_render_frame` function

🏠 The setup of the environment is important to find the shortest path to the goal using different RL algorithms.

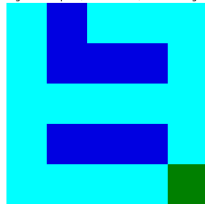
What you will obtain after running each environments...

Light blue=path, Blue=holes, Green=target



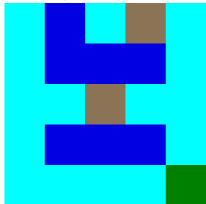
(a) Simplest maze

Light blue=path, Blue=holes, Green=target



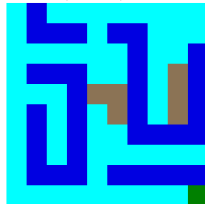
(b) Simple maze

Green=Goal, Blue=Hole, Brown=Mountain



(c) Complex maze

Green=Goal, Blue=Hole, Brown=Mountain



(d) Larger complex maze

What to consider in solutions of the different environments?

- Importance of `env.reset()`
- Increase the `max_steps`
- Increase or decrease the number of episodes depending on the size of the maze
- Prevent infinite loops: exponential decay of epsilon values for small and large maze
- Importance of rewards affectation in RL applications and mean episode return convergence

How to use the environments code?

Run the different environments with the preferred file.

- Understand the `__init__`, `reset()`, `step()` and `render()` functions, and the additional ones from the `GridWorldEnv` class.
- Understand how the `truncated` and `terminated` states are affected, use `env_check` with relevant options.
- Adapt one of the environment code and build your own maze.
- Test your environment.

How to use the solutions code?

Run the different solutions with the preferred file.

- Understand how a change in the rewards will affect the Q-values and the path taken to reach the target.
- Change a few parameters in the solutions files (e.g. the `seed` value, the initial agent location, the rewards, the hyperparameters of the ϵ -greedy policy)
- Try to obtain a convergence of the mean episode return

🕒 Complement the code with moving obstacles in the *custom environment and solutions file*.

An aerial photograph of a town, likely Innsbruck, Austria, with a large, prominent mountain peak in the background under a clear blue sky. The town is nestled in a valley, surrounded by green fields and dense forests. Several large, light-colored buildings are visible, including a prominent one on the left. A winding path or road cuts through the green fields in the foreground.

Now it's your turn!

🔍 Questions are welcome and the slides will be available in
GitHub soon